

Ex.No:21

Experiment : Write a Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to solve the Towers of Hanoi problem.

Objectives

- To understand recursion in Prolog.
- To move all disks from source to destination.
- To follow the rules of the Towers of Hanoi.

Algorithm

1. Start.
2. Read the number of disks.
3. Move N-1 disks from source to auxiliary.
4. Move the largest disk to destination.
5. Move N-1 disks from auxiliary to destination.
6. Stop.

Program

```
move(1,X,Y,_):-
```

```
    write('Move disk from '),
```

```
    write(X),
```

```
    write(' to '),
```

```
    write(Y),nl.
```

```
move(N,X,Y,Z):-
```

```
    N>1,
```

```
    N1 is N-1,
```

```
    move(N1,X,Z,Y),
```

```
    move(1,X,Y,_),
```

```
    move(N1,Z,Y,X).
```

Sample Input

?- move(3,left,right,center).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl').
compiling C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Towers of Hanoi.pl compiled, 12 lines read - 1423 bytes written, 5 ms

yes
| ?- move(3,left,right,center).
Move disk from left to right
Move disk from left to center
Move disk from right to center
Move disk from left to right
Move disk from center to left
Move disk from center to right
Move disk from left to right
true ? |
```

Result

Thus, the Towers of Hanoi problem was implemented successfully using Prolog.

Ex.No:22

Experiment 22: Write a Prolog Program to Print Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a bird can fly.

Objectives

- To understand facts and rules.
- To identify birds that can fly.
- To display the result.

Algorithm

1. Start.
2. Store bird names.
3. Define birds that can fly.
4. Query the bird.
5. Display whether it can fly.
6. Stop.

Program

bird(parrot).

bird(crow).

bird(pigeon).

bird(peacock).

bird(ostrich).

bird(penguin).

can_fly(parrot).

can_fly(crow).

can_fly(pigeon).

can_fly(peacock).

Sample Input

?- can_fly(parrot).

Another Input

?- can_fly(ostrich).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Bird Can Fly or Not.pl compiled, 11 lines read - 930 bytes written, 5 ms
| ?- can_fly(parrot).

yes
| ?- can_fly(ostrich).

no
| ?- |
```

Result

Thus, the Prolog program to determine whether a bird can fly was implemented successfully.

Ex.No:23

Experiment 23: Write a Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree.

Objectives

- To understand family relationships.
- To implement parent relationships.
- To retrieve family information.

Algorithm

1. Start.
2. Store parent relationships.
3. Define father and mother.
4. Query the relationships.
5. Display the result.
6. Stop.

Program

```
father(ramesh,ram).
```

```
father(ramesh,sita).
```

```
mother(latha,ram).
```

```
mother(latha,sita).
```

```
parent(X,Y):-father(X,Y).
```

```
parent(X,Y):-mother(X,Y).
```

Sample Input

```
?- parent(ramesh,ram).
```

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/sum.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/sum.pl compiled, 5 lines read - 826 bytes written, 7 ms
| ?- consult('C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl').
compiling C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Family Tree.pl compiled, 8 lines read - 823 bytes written, 4 ms

yes
| ?- parent(ramesh,ram).

true ?

yes
| ?-
```

Result

Thus, the family tree was implemented successfully using Prolog.

Ex.No:24

Experiment : Write a Prolog Program to Suggest Dieting System Based on Disease

Aim

To write a Prolog program to suggest a suitable diet based on the disease.

Objectives

- To understand Prolog facts.
- To store disease and diet information.
- To suggest an appropriate diet.

Algorithm

1. Start.
2. Store diseases and diets.
3. Enter the disease.
4. Display the recommended diet.
5. Stop.

Program

```
diet(diabetes,'Low Sugar Diet').
```

```
diet(bp,'Low Salt Diet').
```

```
diet(obesity,'Low Fat Diet').
```

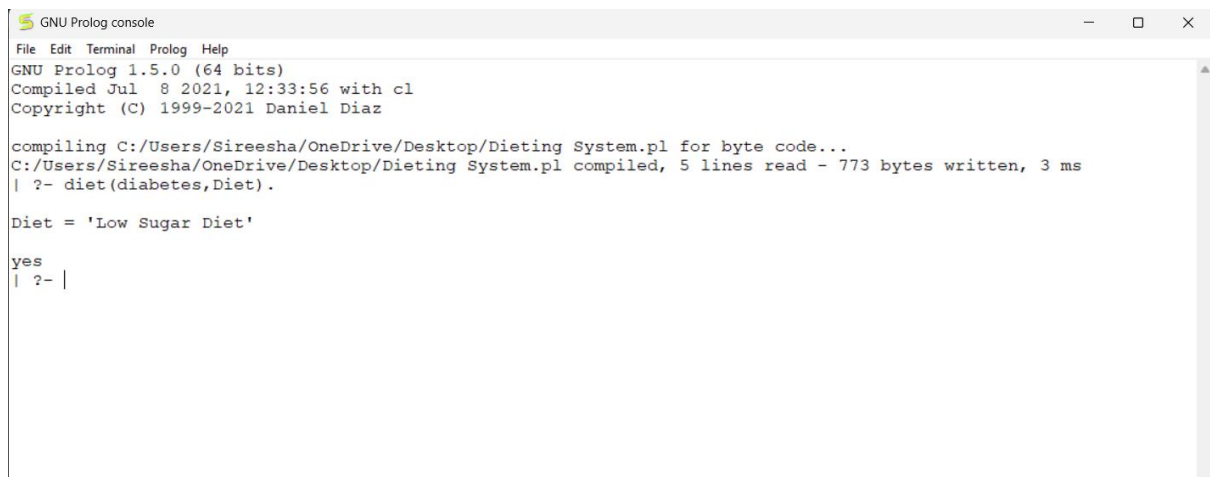
```
diet(fever,'Liquid Diet').
```

```
diet(anemia,'Iron Rich Diet').
```

Sample Input

?- diet(diabetes,Diet).

Sample Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Dieting System.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Dieting System.pl compiled, 5 lines read - 773 bytes written, 3 ms
| ?- diet(diabetes,Diet).

Diet = 'Low Sugar Diet'

yes
| ?- |
```

Result

Thus, the Prolog program to suggest a diet based on disease was implemented successfully.

Ex.No:25

Experiment : Write a Prolog Program to Implement Monkey Banana Problem

Aim

To write a Prolog program to solve the Monkey Banana problem.

Objectives

- To understand the Monkey Banana problem.
- To represent actions using Prolog.
- To determine whether the monkey can reach the banana.

Algorithm

1. Start.
2. Define the monkey's initial position.
3. Define the box position.
4. Move the monkey to the box.
5. Push the box under the banana.
6. Climb the box.
7. Grab the banana.
8. Stop.

Program

at(monkey,door).

at(box>window).

at(banana,center).

move(monkey,door>window).

push(box>window,center).

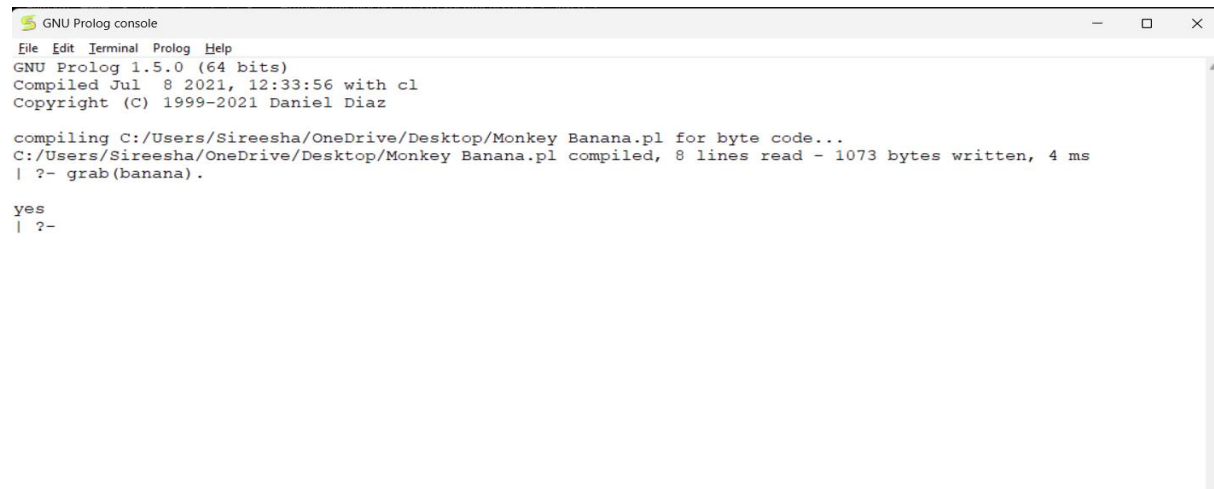
climb(monkey).

grab(banana).

Sample Input

?- grab(banana).

Output

A screenshot of a GNU Prolog console window. The window has a title bar with the text "GNU Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Terminal", "Prolog", and "Help". The main text area shows the following output:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Monkey Banana.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Monkey Banana.pl compiled, 8 lines read - 1073 bytes written, 4 ms
| ?- grab(banana).

yes
| ?-
```

Result

Thus, the Monkey Banana problem was implemented successfully using Prolog.

Ex.No:26

Experiment 26: Write a Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write a Prolog program to store the names of fruits and their colors and retrieve the information using backtracking.

Objectives

- To understand backtracking in Prolog.
- To create a fruit and color database.
- To retrieve fruit names based on color.
- To display all possible solutions using backtracking.

Algorithm

1. Start.
2. Store fruits and their corresponding colors as facts.
3. Enter a query with a fruit name or color.
4. Prolog searches the database.
5. If multiple solutions exist, use backtracking to display all of them.
6. Stop.

Program (Prolog)

% Fruit and Color Database

fruit(apple, red).

fruit(banana, yellow).

fruit(grapes, green).

fruit(mango, yellow).

fruit(orange, orange).

fruit(guava, green).

Sample Input 1

?- fruit(apple, Color).

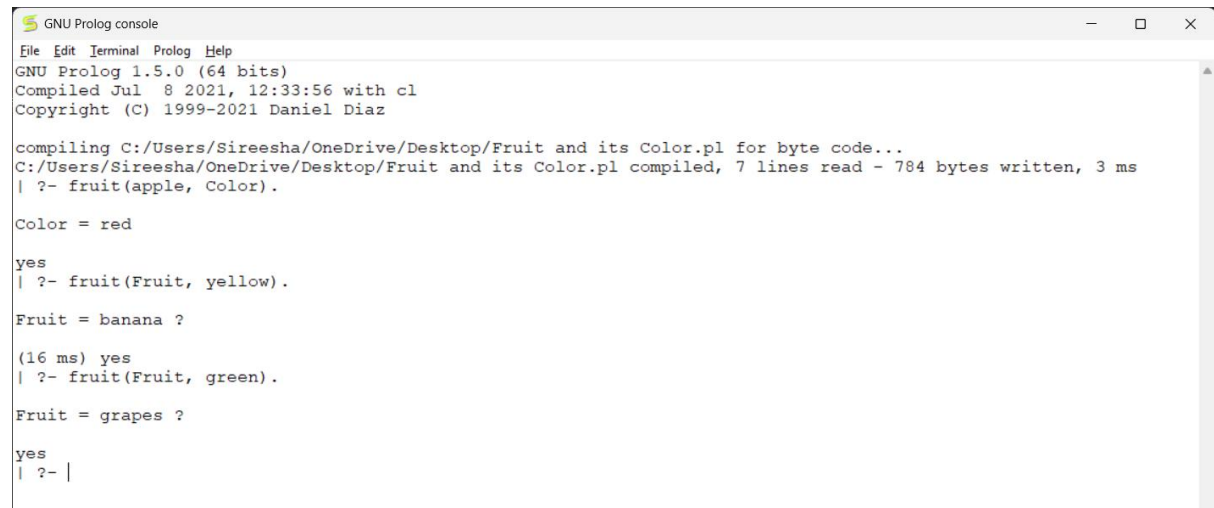
Sample Input 2

?- fruit(Fruit, yellow).

Sample Input 3

?- fruit(Fruit, green).

Output



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/Sireesha/OneDrive/Desktop/Fruit and its Color.pl for byte code...
C:/Users/Sireesha/OneDrive/Desktop/Fruit and its Color.pl compiled, 7 lines read - 784 bytes written, 3 ms
| ?- fruit(apple, Color).

Color = red

yes
| ?- fruit(Fruit, yellow).

Fruit = banana ?

(16 ms) yes
| ?- fruit(Fruit, green).

Fruit = grapes ?

yes
| ?- |
```

Result

Thus, the Prolog program for Fruit and Its Color Using Backtracking was successfully implemented. The program retrieved all matching fruits for a given color using Prolog's backtracking mechanism.